

Questionário Técnico - Quality Assurance Junior I

Candidato: Pedro Paulo

Processo Seletivo: Port Louis

Data: Maio de 2026

Você recebeu um card para desenvolver.

1. Qual sequência de comandos você executa desde atualizar o projeto até começar a desenvolver?

Resposta: Dado que já estou no VScode, Primeiro verifico se estou na Branch correta com **git checkout develop** e então atualizo para versão mais atualizada da branch usando **git pull origin develop**. Com a branch atualizada crio minha branch de trabalho seguindo o padrão do time, por exemplo: **git checkout -b feature/nome-da-tarefa**.

Para finalizar a preparação do ambiente, executo o npm install (Ou Yarn) para que qualquer dependência necessária seja baixada no meu ambiente, evitando flaky test

Você tentou fazer merge e houve conflito.

1. Como você identifica os arquivos com conflito? Qual o processo que você segue para resolver?

Resposta: Na hora de executar um Merge e os códigos derem conflitos, primeiro eu comparo os dois códigos, vejo se foi uma alteração específica de outro dev. Caso seja preciso alinhar com ele, qual seria a melhor forma de seguir, para eu não tomar uma decisão precipitada. Então organizamos a branch, removendo os conflitos e deixando apenas as linhas corretas. Após organizar localmente, voltamos ao terminal e para sinalizar a resolução executamos **Git add .** ou **Git add + (arquivo específico)** e finalizo com o commit de merge mas antes de seguir para o Push, precisamos executar um Smoke test, para garantir que a integração não afetou a aplicação em si.

Sua feature está pronta

1. Quais comandos você executa para enviar seu código?

Resposta: Dado que finalizei a Feature e testei localmente, para enviar meu código, primeiro executo um **git status** para fazer mais uma revisão das alterações e que não estou enviando arquivos desnecessários.

Logo após executo o **Git add .** ou **Git add + (arquivo específico)** e crio o commit com uma mensagem clara e bem descritiva usando **git commit -m "feat: descrição da funcionalidade"** seguindo as preferências de mensagem do time

Antes de realmente enviar, também confirmo que meu código está integrado com as últimas mudanças do dia executando **git pull origin develop**.

Para finalizar, executo o **git push origin feature/minha-task** para subir o código e colocar em análise de PR (Revisão).

Um bug apareceu em produção e você precisa investigar.

1. Como você encontra em qual commit o problema começou?

Resposta: Para investigar a origem de um bug em produção, eu sigo cada degrau por dificuldade, começando com **git log** filtrando pelos arquivos ou pastas relacionados a funcionalidade que quebrou, podendo encontrar a origem de **Quem alterou e o que alterou**.

Caso ainda precise me aprofundar, filtramos por versão e investigamos até qual estava boa e qual começou a quebrar, e investigamos usando o intermédio de versão “boa” e versão “quebrada”.

Se mesmo assim precisar de mais ferramentas para encontrar o problema, uso o **git bisect**. Filtrando além das versões, para filtrar o Commit entre “commit bom” e “ruim”.

Após encontrar o problema, trazendo o contexto e comunicação clara para o dev entender rapidamente, podendo aplicar o hotfix

O time entregou uma feature na branch develop.

1. O que você faz antes de iniciar os testes nessa branch?

Resposta:

Antes de iniciar os testes na develop, eu limpo meu ambiente local. Primeiro, faço o **git checkout** para a branch e executo o **git pull**. Em seguida, rodo o comando para instalar as dependências (**npm install**), pois se o desenvolvedor adicionou uma nova biblioteca e eu não a tiver, posso acabar perdendo tempo com falsos positivos de ambiente e não por bug no código. Também dou uma olhada rápida no histórico de commits para entender o que exatamente foi alterado e concentrar os casos de teste no que realmente importa.

2. Como você garante que sua base local está atualizada?

Resposta: Garanto a atualização através do comando **git pull origin develop**. Para ter certeza absoluta, dou uma olhada no **Hash do último commit** ou a mensagem de commit no terminal para comparar com a que está no **GITHUB**. Se os logs estiverem alinhados, sei que estou testando a versão correta.

Você identificou que o comportamento em beta está diferente do develop.

1. Como você investiga se é problema de código ou de deploy? Quais evidências você coleta?

Resposta: Para investigar essa diferença, primeiro preciso estar atento ao código e a infraestrutura.

Primeiro verificar se o commit é exatamente o mesmo que foi aprovado em develop. Às vezes o deploy foi feito em uma branch desatualizada.

Coleta prints do console do navegador (erros 404/500), logs da pipeline de deploy e o log de rede para comparar as respostas das APIs entre os dois ambientes.

Um deploy falhou na pipeline.

1. O que você analisa primeiro? Você consegue identificar em qual etapa falhou (build, teste, deploy)? Qual sua ação após identificar a falha?

Resposta: Minha primeira ação é verificar os **logs da Pipeline**. Posso dividir o processo em etapas, primeiro verifico Erro de sintaxe ou dependências do código, Falha funcional ou regressão detectada e também verificar se houve problema de infraestrutura ou permissões de servidor.

Ao identificar a falha, envio o log ou print comunicando com contexto o squad.

Então se for erro de Build ou teste, marco o desenvolvedor da tarefa. Caso o erro seja de deploy, sinalizo o responsável pela Infra, informando que o ambiente está instável, até resolvermos.

Meu objetivo é garantir que o bloqueio seja resolvido rápido, independentemente de quem seja a responsabilidade, para não atrasar a entrega.

Pensando no fluxo develop > beta > master:

1. Em qual momento você bloqueia uma subida para produção?

Resposta: Eu sempre vou bloquear quando identificar um **Bug Crítico** que afete o fluxo principal do usuário ou quando os teste de regressão apontam que uma funcionalidade antiga parou de funcionar. Além disso, se o comportamento em **Beta não for o esperado** eu notifico o impedimento.

2. O que define que uma entrega está pronta para produção?

Resposta: Uma entrega está pronta quando passa por todas etapas do time com êxito. Após Código ser Revisado, Ser aprovado nos teste automatizados, A nova funcionalidade ser validada manualmente em ambiente de Beta, e todos bugs abertos terem sido tratados.

Com esses critérios atendidos e o aceite do PO, posso considerar o código estável para o Merge em Master.